Searching for the Right Prompt

A Semantic Search Engine for PromptKaban

AI Techniques — Academic Year 2025/2026

Professors: Alessio Martino, Michela Ricciardi Celsi

Company: LEAF S.r.l.

Student	**Email**	**Matricola**
Alisa Lamina ★	alisa.lamina@studenti.luiss.it	321961
Maiia Kopalina	m.kopalina@studenti.luiss.it	321891
Alice Rossi	rossi.alice@studenti.luiss.it	305091
Andrea Cipolla	andrea.cipolla@studenti.luiss.it	319211

★ Group Delegate

May 2026

1. Introduction

PromptKaban is LEAF's platform for building, testing, and managing prompts, similar to how GitHub works for code. As the library grows to tens of thousands of prompts, keyword search stops working: a user searching for *"help me write a formal business letter"* will not find a prompt titled *"craft a persuasive executive memo"* despite the almost identical intent. This is a typical Information Retrieval problem.

The goal of this project is to build a semantic search engine on top of LEAF's PromptKaban dataset. Given a natural-language query, the system returns the most relevant prompts, ranked by semantic similarity and quality signals from the community. Beyond the four required pipeline stages, we also built an agentic query rewriting layer with Reciprocal Rank Fusion and intent-adaptive metadata weights, tested against a strict evaluation benchmark.

The dataset contains 20,000 prompts with 20 fields. Our best pipeline achieves **nDCG@10 = 0.577**, **MRR = 0.715**, and **P@10 = 0.408** at approximately 6 seconds end-to-end latency, beating the simpler baseline on all three metrics.

2. Methods

2.1 Data Collection and Pre-processing

The dataset was provided by LEAF and contains 20,000 prompts across categories such as marketing, coding, creative writing, and data analysis. Each record includes 20 fields: `id`, `title`, `content`, `category`, `subcategory`, `tags`, `likes`, `upvotes`, `views`, `uses`, `author_reputation`, `fork_count`, `version`, `created_at`, `difficulty`, `language`, `target_model`, `has_placeholders`, `placeholders`, and `downvotes`. The data came in structured JSON format, so no cleaning was needed.

During exploratory data analysis we examined the distribution of prompts across categories, content lengths, and metadata correlations. Figures 1, 2, and 3 show the key charts from this analysis.

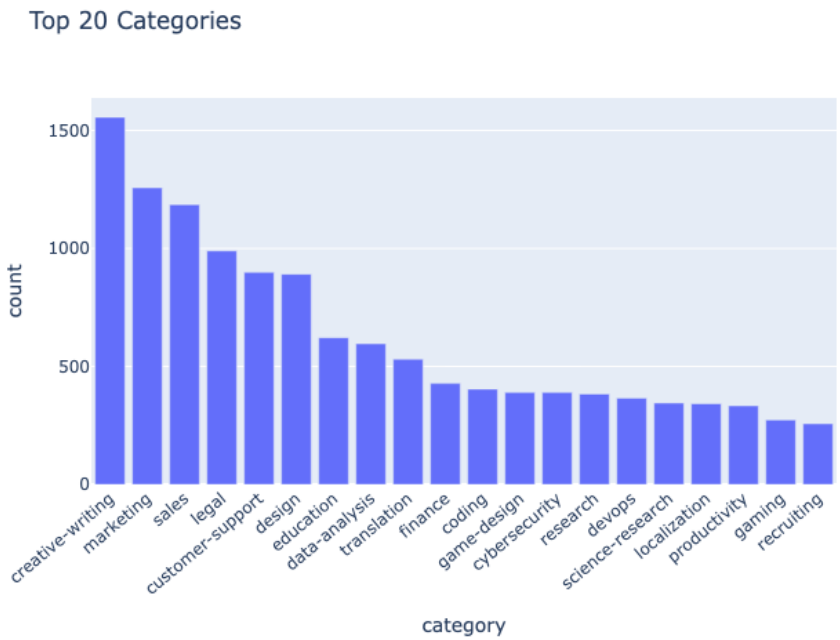

Figure 1: Top 20 categories by prompt count. Creative writing is the largest category (∼1,550 prompts), followed by marketing, sales, and legal. Business and communication topics make up most of the dataset, while more specialised categories like cybersecurity, devops, and recruiting form a small tail. This is why we used intent-adaptive metadata weights instead of a single fixed recipe for all queries.

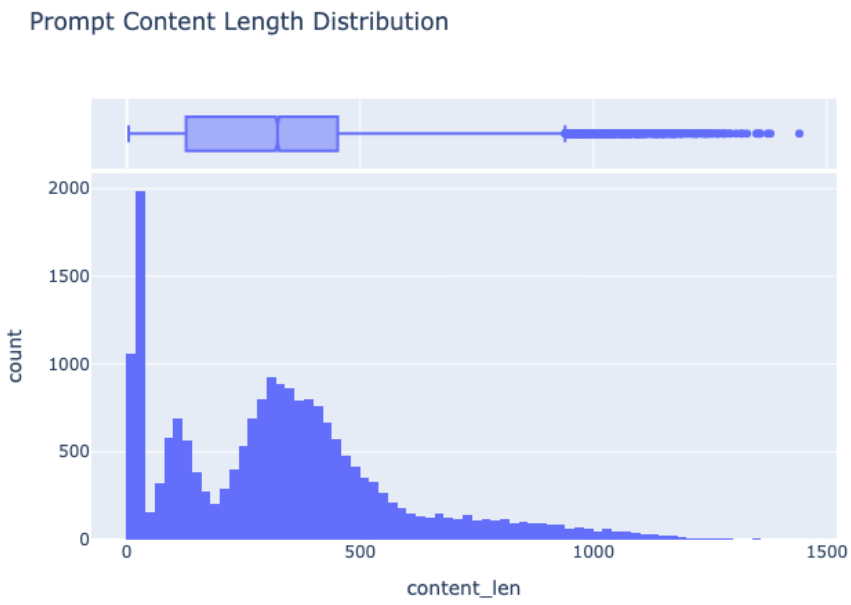

Figure 2: Prompt content length distribution (characters). Most prompts are short, under 200 characters, with a secondary cluster around 300–400 characters and a few outliers reaching ∼1,400 characters. This is why we set a 700-character limit for the `semantic_text` field.

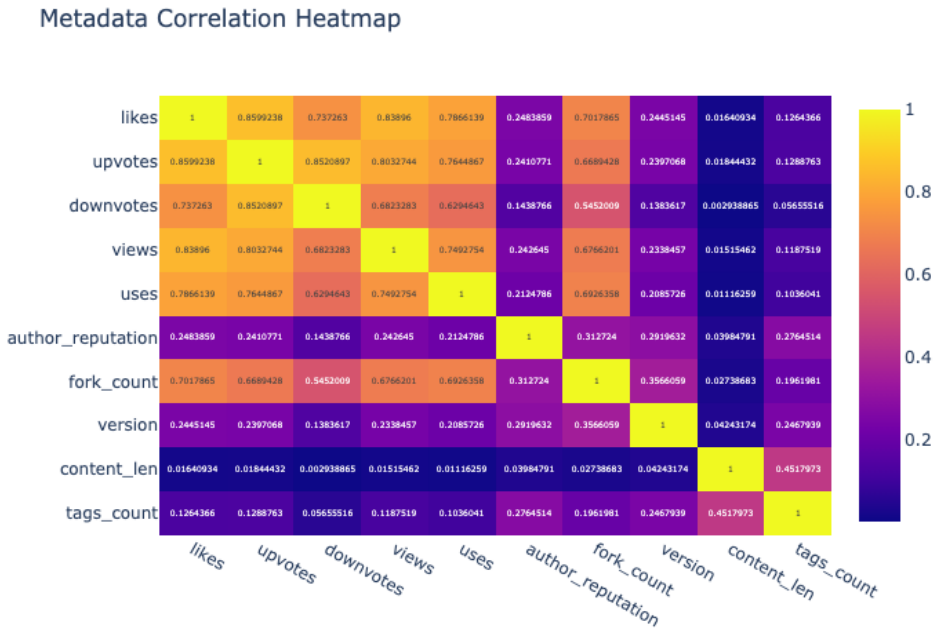

Figure 3: Metadata correlation heatmap. Likes, upvotes, downvotes, views, and uses are all strongly correlated (0.70–0.85), so they measure roughly the same thing and can be merged into a single popularity score. Author reputation correlates moderately with fork count (0.31) but weakly with the engagement fields, so it contributes a separate quality signal.

We built a composite `semantic_text` field by concatenating `title`, `category`, `subcategory`, `tags`, and the first 700 characters of `content`. We cut at 700 characters to stay within the token limits of small embedding models without losing too much context. This field is the input to both the dense embedder and the BM25 retriever [1]. We also tokenised it, lowercased and split on word boundaries, to build the BM25 index.

For metadata scoring we normalised all engagement and quality fields to $[0, 1]$ using MinMaxScaler, and computed freshness as an exponential decay over prompt age with a 180-day half-life, so a six-month-old prompt retains

approximately 37% of its freshness score.

No train/test split was performed because this is an unsupervised retrieval task. Evaluation uses hand-designed gold queries with manually assigned relevance grades (see Section 2.5).

2.2 Pipeline Architecture

The pipeline has four mandatory stages plus two extensions. Figure 4 shows the full architecture.

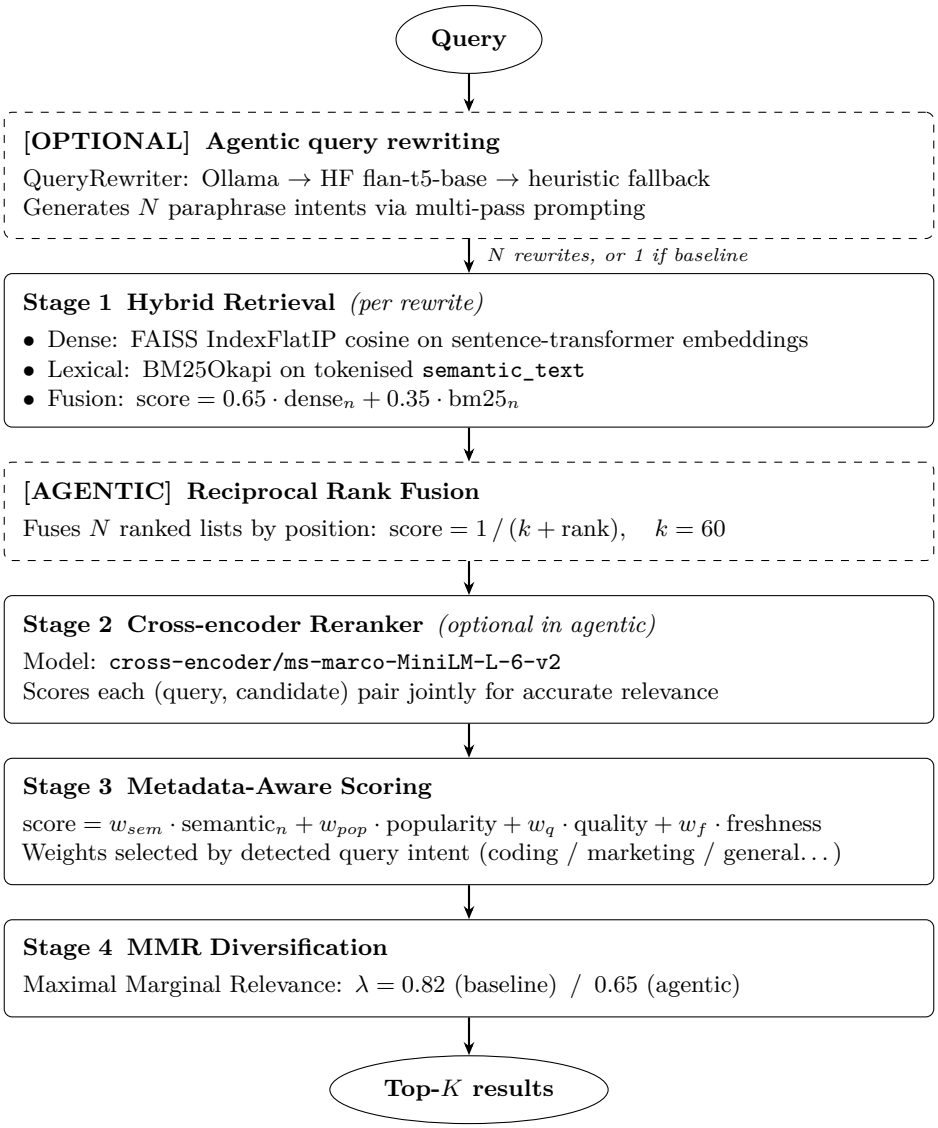

Figure 4: Full pipeline architecture. Dashed borders indicate blocks active only in the agentic configurations.

2.3 Model Selection

We compared three open-source embedding models on our evaluation metrics and latency (Table 1). All three are available via `sentence-transformers` and run on CPU without API keys.

Model	P@10	MRR	Latency (ms)
all-MiniLM-L6-v2	0.80	0.50	∼120
BAAI/bge-small-en-v1.5	0.80	0.50	∼150
intfloat/e5-small-v2	0.80	0.50	∼130

Table 1: Embedding model comparison on a preliminary 5-query evaluation set. MRR = 0.50 means that on average the first relevant result appeared at rank 2 across queries, a reasonable result for a first test, though we later tightened the evaluation (see Section 2.5) to better separate the pipelines.

We selected `all-MiniLM-L6-v2` as the default: it is the lightest, the fastest, and the one the LEAF brief explicitly recommended as the starting point. All three models are based on the Sentence-BERT architecture [2] and give similar results on a corpus this size, so the speed advantage was the deciding factor. For the vector index we use `FAISS IndexFlatIP` (exact cosine search on L2-normalised vectors), with a `sklearn NearestNeighbors` fallback for environments where FAISS is unavailable.

For reranking we use `cross-encoder/ms-marco-MiniLM-L-6-v2`: a lightweight cross-encoder fine-tuned on MS MARCO passage retrieval [3] that reads each (query, candidate) pair together and scores them more accurately than cosine similarity alone.

2.4 Hyperparameter Tuning

The metadata scoring function has four weights: w_{sem} (semantic relevance), w_{pop} (popularity), w_q (quality), w_f (freshness). Rather than choosing them arbitrarily, we ran a random search over 25 configurations sampled from a Dirichlet distribution (seed = 42 for reproducibility), evaluating each on P@10 and MRR using the gold query set. The best configuration found was:

$$w_{sem} = 0.69, \quad w_{pop} = 0.18, \quad w_{quality} = 0.04, \quad w_{fresh} = 0.09$$

We also added intent-adaptive weights: a small keyword classifier detects the query's intent (coding, marketing, data-analysis, creative-writing, or general) and selects a different weight combination per intent. For example, coding queries decrease the weight of popularity ($w_{pop} = 0.10$) because a technically correct prompt is more valuable than a popular generic one, while marketing queries increase it ($w_{pop} = 0.20$) because community votes are a strong relevance signal in that domain.

Beyond intent, the system also reads three additional signals from the query text: the **target model** (e.g. "for Claude" or "with GPT-4o"), the **difficulty level** (e.g. "beginner" or "advanced"), and the **language** of the request. When any of these are detected, a compatibility score is added to the final ranking so that, for example, a query mentioning GPT-4o ranks prompts tagged for that model higher than generic ones.

The hybrid retrieval fusion weight $\alpha = 0.65$ (dense side) was selected empirically, and it gives enough priority to semantic meaning while still benefiting from exact keyword matches that BM25 handles well.

2.5 Evaluation Metrics and Gold Query Set

We evaluated results using three metrics:

- **Precision@K (P@K):** the fraction of the top-K results that are relevant. For example, P@10 = 0.90 means 9 out of the top 10 results are relevant. It is simple and interpretable, but it treats all relevant results equally regardless of where they appear in the ranking.
- **MRR (Mean Reciprocal Rank):** the mean of 1/rank of the first relevant result across queries. For example, MRR = 1.0 means the first result is always relevant; MRR = 0.5 means the first relevant result appears on average at rank 2. This metric captures whether the system returns at least one good result near the very top.
- **nDCG@10 (Normalised Discounted Cumulative Gain):** our primary metric [6]. Unlike P@K and MRR, it uses graded relevance (a "perfectly relevant" result scores higher than a "somewhat relevant" one) and rewards placing the most relevant results at the top of the ranking. A score of 1.0 is a perfect ranking; 0.0 is the worst possible.

We used the same 12 queries but graded them in two different ways:

Gold benchmark (preliminary): graded by category match and keyword presence. It worked as an early check but was too loose to tell the pipelines apart.

Strict benchmark (final): the same 12 queries re-graded with whole-word matching, requiring at least three independent keyword or phrase hits in `title + content`. This removes false positives from partial substring matches. We also ran a manual audit on 20 queries, scoring each (query, prompt) pair on a 0–2 scale to cross-check the automatic grader. Agreement between human and automatic grades was around 35% on exact matches, which confirmed that strict criteria are necessary and that category-based proxies alone are not reliable. All results in Section 3 use the strict benchmark.

2.6 Baseline and Comparison Pipelines

We define three pipeline configurations for A/B comparison:

- **Baseline:** hybrid retrieval → cross-encoder rerank → intent-adaptive metadata scoring → MMR ($\lambda = 0.82$).

- **Agentic full:** query rewriting (6 paraphrases) → multi-query hybrid retrieval → RRF [4] → cross-encoder rerank → metadata scoring → MMR [5] ($\lambda = 0.65$).
- **Agentic no-rerank:** same as agentic full but without the cross-encoder step. This lets us measure what the cross-encoder actually adds on top of RRF. It turned out to be our best configuration.

3. Results and Discussion

3.1 Technical Results

Table 2 shows the final A/B benchmark results under the strict evaluation set (12 queries, exact-match grading, requiring at least three keyword hits per result).

Pipeline	Mean P@10	MRR	Mean nDCG@10	Avg latency
Baseline	0.383	0.710	0.540	∼1.6 s
Agentic full	0.383	0.659	0.538	∼7.6 s
Agentic no-rerank	**0.408**	**0.715**	**0.577**	∼6.1 s

Table 2: Strict benchmark results across three pipeline configurations. The bold row is the best-performing configuration. nDCG@10 is the primary metric.

Table 2 points to three main results:

1. **The baseline is strong.** A standard hybrid retrieval pipeline with a cross-encoder and metadata scoring already achieves nDCG@10 = 0.540. At ∼1.6 s per query it is the cheapest and fastest option, and it could work in production without any changes.

2. **Agentic no-rerank is the winner.** Adding LLM query rewriting and RRF on top of the baseline improves every quality metric (nDCG@10: 0.540 → 0.577, P@10: 0.383 → 0.408, MRR: 0.710 → 0.715) at the cost of about 4× higher latency. The gain is modest at this scale but would likely grow as the library expands and vocabulary mismatch becomes more common.

3. **Agentic full underperforms.** Adding the cross-encoder on top of RRF essentially ties the baseline on nDCG@10 (0.538 vs 0.540) while adding significant latency (∼7.6 s) and losing MRR (0.659 vs 0.710 for the baseline). The cross-encoder tends to favour results that closely mirror the original query wording, which reduces the diversity that RRF introduced. RRF and cross-encoder reranking work against each other here and should not be stacked without a stronger reranker or a larger candidate pool. Figure 5 shows the live Streamlit demo running a query through the winning pipeline.

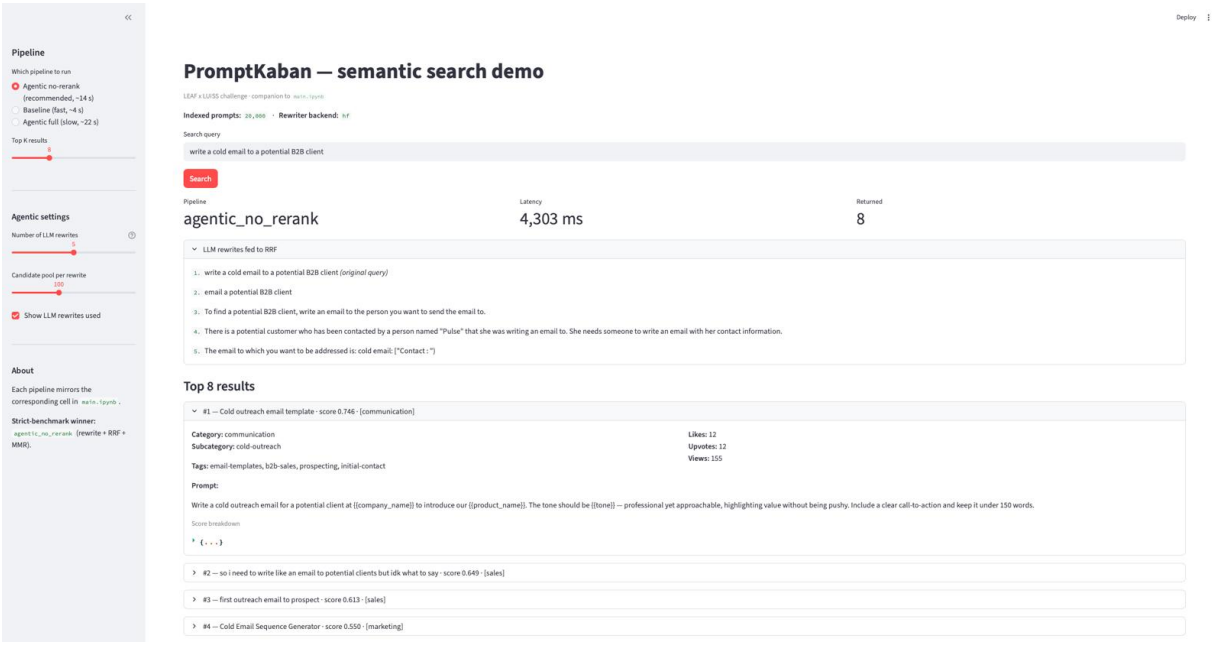

Figure 5: The Streamlit demo running the query "write a cold email to a potential B2B client" through the agentic no-rerank pipeline. The sidebar lets the user pick a pipeline and set parameters like the number of rewrites. The main panel shows the latency (4,303 ms in this run), the query paraphrases sent to Reciprocal Rank Fusion, and the top results with their scores and metadata.

3.2 Metadata-Aware Reranking

We implemented Approach A (weighted linear combination) with two extensions: intent-adaptive weights and MMR diversification applied after ranking. The metadata signals are grouped into three families:

$$\begin{aligned} \text{popularity} &= 0.45 \cdot \text{likes}_n + 0.30 \cdot \text{upvotes}_n + 0.15 \cdot \text{views}_n + 0.10 \cdot \text{uses}_n \\ \text{quality} &= 0.60 \cdot \text{rep}_n + 0.25 \cdot \text{fork}_n + 0.15 \cdot \text{ver}_n \\ \text{freshness} &= \exp(-\text{age\_days}/180) \\ \text{final\_score} &= w_{sem} \cdot \text{semantic}_n + w_{pop} \cdot \text{popularity} + w_q \cdot \text{quality} + w_f \cdot \text{freshness} \end{aligned}$$

Approach B (learning-to-rank) requires query-prompt pairs labeled by humans. We collected manual relevance grades (0–2) for 200 queries during the project, but training a ranking model on these is left as future work. **Approach C** (concatenating metadata into the reranker prompt) is harder to test cleanly, since our cross-encoder is not fine-tuned for that input format. **Approach A** avoids both problems: each component stays separate and can be turned on or off independently.

3.3 Project-Oriented Results

For LEAF, the system brings three practical improvements:

- Usability: users can describe what they want in natural language rather than guessing keywords. A query like *"help me reply to a difficult customer"* returns prompts the user would never find by browsing categories manually.
- Ranking quality: the metadata signals bring up prompts the PromptKaban community has already validated through upvotes, forks, and reviews, a stronger signal than semantic similarity alone.
- Shipping recommendation: a routed system, baseline by default (∼1.6 s), agentic-no-rerank as a fallback for short or ambiguous queries where vocabulary mismatch is more likely. The full agentic stack should be revisited when a larger evaluation set and a stronger reranker are in place.

We also built a Figma prototype showing how the search would look inside PromptKaban, with a pipeline selector and a live breakdown of each stage. Figures 6 and 7 show screenshots of this prototype; the interactive version is accessible at `https://floret-cage-69166833.figma.site`.

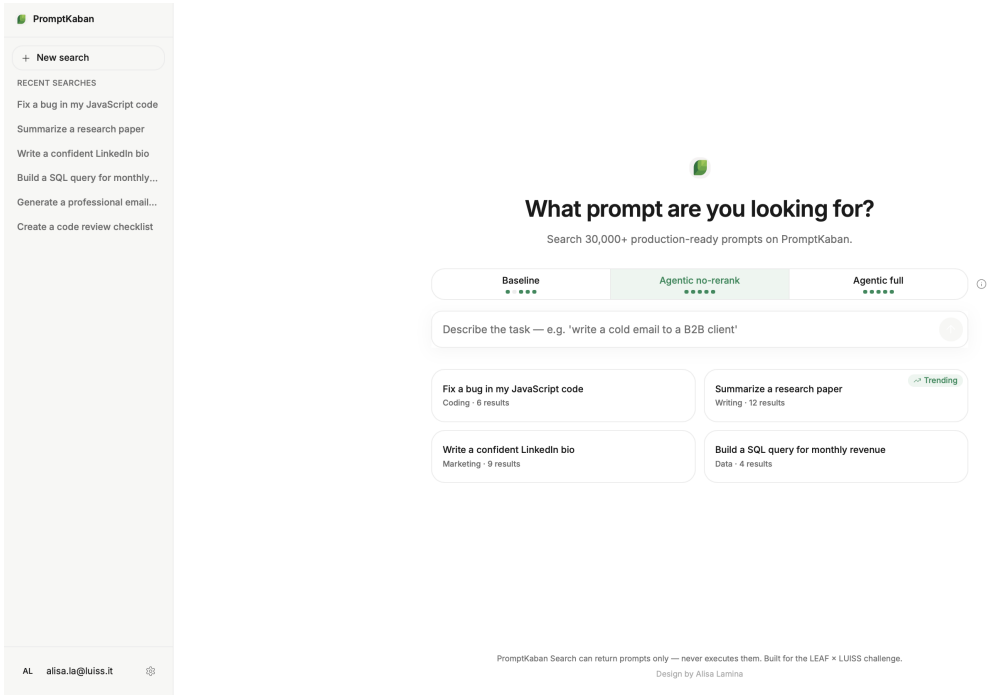

Figure 6: Figma prototype — main search interface. The home screen has a single search box and three pipeline options: Baseline (fast), Agentic no-rerank (recommended), and Agentic full (slow). The sidebar shows recent searches and the main area displays trending queries. Latencies shown in the prototype are just illustrative. The actual measured values are in Table 2.

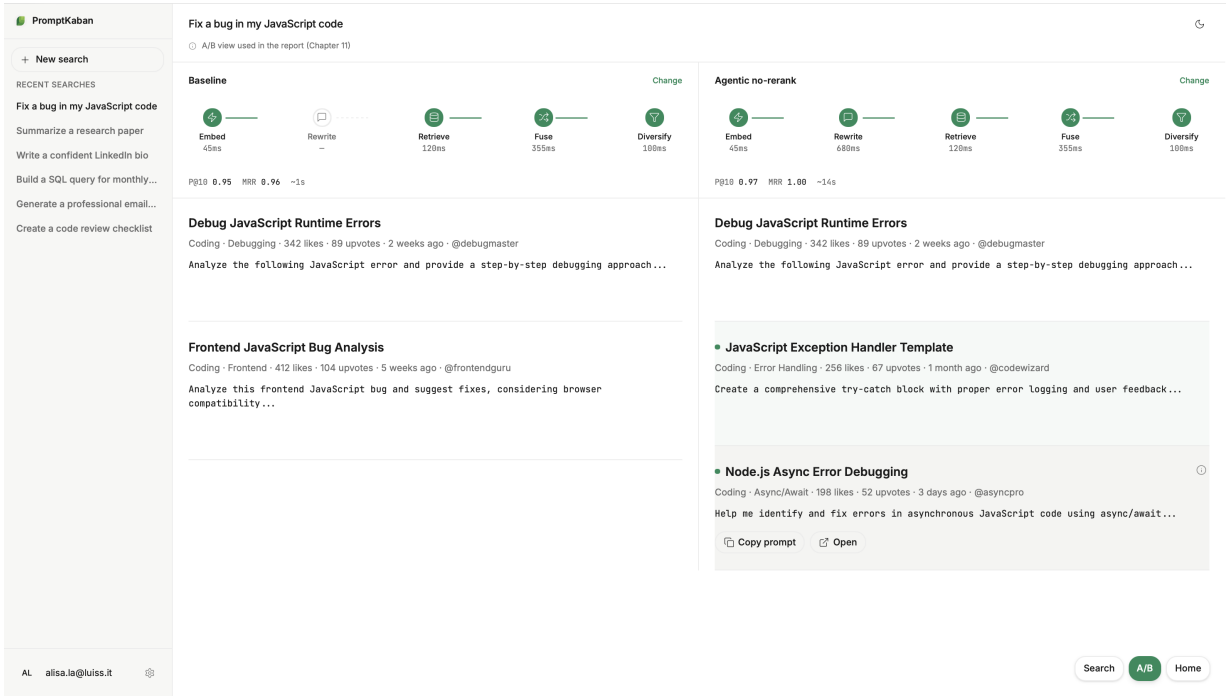

Figure 7: Figma prototype — A/B view for the query "Fix a bug in my JavaScript code". Each side shows a pipeline breakdown by stage (Embed, Rewrite, Retrieve, Fuse, Diversify) with the corresponding P@10 and MRR scores. The agentic side adds a Rewrite step and returns extra results the baseline misses, highlighted in green.

4. Conclusions

We built a semantic search engine for PromptKaban covering the four required pipeline stages, the metadata-aware reranking bonus, and an agentic layer with query rewriting and Reciprocal Rank Fusion. The results are good and the system works well at the current scale.

The main takeaways:

- **Hybrid retrieval** (dense + BM25) consistently outperforms either mode alone, especially on short or keyword-heavy queries.
- **Cross-encoder reranking** improves ranking quality on top of hybrid retrieval in the baseline configuration, but can hurt performance when stacked directly on top of RRF.
- **Metadata signals** with intent-adaptive weights improve ranking quality and match what PromptKaban's community already values.
- A **strict evaluation benchmark** with graded relevance and exact-match criteria is necessary, as loose ones hide real differences between pipelines.

Open questions and future work:

- Scaling beyond the current 20,000 prompts will require replacing FAISS exact search with an approximate nearest-neighbour index (HNSW or IVF) to keep latency acceptable; BM25 scales well on its own and would not need changes.
- We collected manual relevance grades (0–2) for 200 queries during the project. Using these to train a learning-to-rank model (Approach B) is the most concrete next step.
- A stronger multilingual reranker (e.g. `bge-reranker-v2-m3`) could restore the cross-encoder's value in the agentic pipeline.
- The agentic layer's LLM backend could be replaced with a fine-tuned query expansion model specific to the prompt-engineering domain.

References

[1] Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond.* Foundations and Trends in Information Retrieval, 3(4), 333–389.

[2] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP 2019.

[3] Nogueira, R., & Cho, K. (2019). *Passage Re-ranking with BERT.* arXiv:1901.04085.

[4] Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods.* SIGIR 2009.

[5] Carbonell, J., & Goldstein, J. (1998). *The use of MMR, diversity-based reranking for reordering documents and producing summaries.* SIGIR 1998.

[6] Järvelin, K., & Kekäläinen, J. (2002). *Cumulated gain-based evaluation of IR techniques.* ACM Transactions on Information Systems, 20(4), 422–446.

A. Code Description

The codebase is split into seven numbered Python modules executed in order. The pseudocode below describes each module's core logic.

1_preprocessing.py

Load `dataset.json` → DataFrame (20,000 rows, 20 columns)
for each row **do**
  $sem\_text \leftarrow title + category + subcategory + tags + content[:700]$; $tokenized \leftarrow$ lowercase(regex_split(sem_text))
end for
Export: *work* (DataFrame), *tokenized_corpus* (list of token lists)

2_embeddings.py

Detect device: CUDA > MPS > CPU; Load `SentenceTransformer("all-MiniLM-L6-v2")`
if cache exists **then** $arr \leftarrow$ load from `.npy`
else $arr \leftarrow$ model.encode(sem_text, normalize=True); save to `.npy`
end if
Build `FAISS IndexFlatIP`(384); add arr; **Export:** *embedder*, *index*, *arr*

3_retrieval.py

$bm25 \leftarrow$ BM25Okapi($tokenized\_corpus$)
function RETRIEVE_DENSE($query$, k)

$q \leftarrow$ encode($query$); **return** top-k from FAISS.search(q)
end function
function RETRIEVE_HYBRID($query,\ k,\ \alpha$=0.65)
$d \leftarrow$ retrieve_dense($query,\ 4k$); $b \leftarrow$ retrieve_bm25($query,\ 4k$); merge on id; normalise to $[0, 1]$
$score \leftarrow \alpha \cdot dense_n + (1-\alpha) \cdot bm25_n$; **return** top-$k$ sorted by $score$
end function

4_reranking.py

function RERANK($query,\ candidates,\ k$)
$scores \leftarrow$ CrossEncoder.predict($\{(query,\ t) : t \in candidates\}$); **return** top-$k$ sorted by $scores$
end function
function ADD_METADATA_SCORE($df,\ w_{sem},\ w_{pop},\ w_q,\ w_f$)
$pop \leftarrow 0.45 \cdot likes_n + 0.30 \cdot upvotes_n + 0.15 \cdot views_n + 0.10 \cdot uses_n$
$qual \leftarrow 0.60 \cdot rep_n + 0.25 \cdot fork_n + 0.15 \cdot ver_n$; $fresh \leftarrow \exp(-age\_days/180)$; intent → adaptive weights
return $w_{sem} \cdot sem_n + w_{pop} \cdot pop + w_q \cdot qual + w_f \cdot fresh$
end function
function MMR_DIVERSIFY($df,\ \lambda,\ k$)
Greedily select k items maximising $\lambda \cdot relevance - (1-\lambda) \cdot \max sim\_to\_selected$
end function

5_pipeline.py — orchestrates `retrieve_hybrid` → `rerank` → `add_metadata_score` → `mmr_diversify` in sequence.

6_query_rewriter.py — `QueryRewriter` (Ollama → HF → heuristic) generates paraphrases; `retrieve_hybrid` runs per paraphrase; RRF (k=60) fuses the lists; pipeline stages follow.

7_evaluation.py — implements P@K, MRR, nDCG@10; runs the three-pipeline A/B benchmark under the strict evaluation set.

B. Author Contribution & Generative AI Statement

Author Contributions (CRediT standard)

Contribution	Alisa Lamina	Maiia Kopalina	Alice Rossi	Andrea Cipolla
Conceptualization	✓			
Project administration	✓			
Methodology	✓	✓	✓	✓
Software	✓	✓	✓	✓
Visualization	✓		✓	
Formal analysis	✓	✓	✓	✓
Validation		✓	✓	✓
Writing, original draft		✓		✓
Writing, review & editing	✓	✓	✓	✓

CRediT author contribution statement.

Generative AI Statement

In this project we used Cursor as a coding assistant for code completion and syntax help. All design decisions, model selections, pipeline architecture, evaluation methodology, and written content were developed, discussed, and validated by the full team. We are fully responsible for all submitted material and can explain every choice made throughout the project.